

Import Libraries

```
import os
import pandas as pd
import pathlib
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import random
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPool2D,
Activation, Dropout, BatchNormalization
from tensorflow.keras import Sequential
```

Import Datasets

```
for dirpath, dirnames, filenames in os.walk("dataset_cleaned"):
    print(f"There are {len(dirnames)} directories and {len(filenames)}
images in '{dirpath}'.")
```

```
There are 2 directories and 2 images in 'dataset_cleaned'.
There are 2 directories and 1 images in 'dataset_cleaned/test'.
There are 0 directories and 1000 images in
'dataset_cleaned/test/male'.
There are 0 directories and 1000 images in
'dataset_cleaned/test/female'.
There are 2 directories and 1 images in 'dataset_cleaned/train'.
There are 0 directories and 3000 images in
'dataset_cleaned/train/male'.
There are 0 directories and 3000 images in
'dataset_cleaned/train/female'.
```

Train and Test Split

```
dataset_train = "dataset_cleaned/train/"
dataset_test = "dataset_cleaned/test/"
```

Declare Class Names

```
data_dir = pathlib.Path(f"{dataset_train}") # turn our training path
into a Python path
class_names = np.array(
    sorted([
        item.name
        for item in data_dir.glob('*')
        if not item.name.startswith('.')
    ])
)

print(class_names)

['female' 'male']
```

Define function to view random image

```
def view_random_image(target_dir, target_class):
    # Setup target directory (we'll view images from here)
    target_folder = target_dir+target_class

    # Get a random image path
    random_image = random.sample(os.listdir(target_folder), 1)

    # Read in the image and plot it using matplotlib
    img = mpimg.imread(target_folder + "/" + random_image[0])
    plt.imshow(img)
    plt.title(target_class)
    plt.axis("off");

    print(f"Image shape: {img.shape}") # show the shape of the image

    return img
```

View a random image

```
img =
view_random_image(target_dir=f"{dataset_train}",target_class="male")

Image shape: (219, 79, 3)
```

A man in a dark jacket and light-colored pants is walking away from the camera, carrying a bag. The image is blurry and appears to be a still from a video.

```
img.shape
# returns (width, height, colour channels)
(219, 79, 3)
```

[illegible]

```
# convert all images to be 224 x 224
```

```
type of problem we're working on
```

```
class_mode="binary", #  
seed=42)
```

```
valid_data = valid_datagen.flow_from_directory(test_dir,  
                                                batch_size=32,  
                                                target_size=(224, 224),  
                                                class_mode="binary",  
                                                seed=42)
```

```
Found 6000 images belonging to 2 classes.
```

```
Found 2000 images belonging to 2 classes.
```

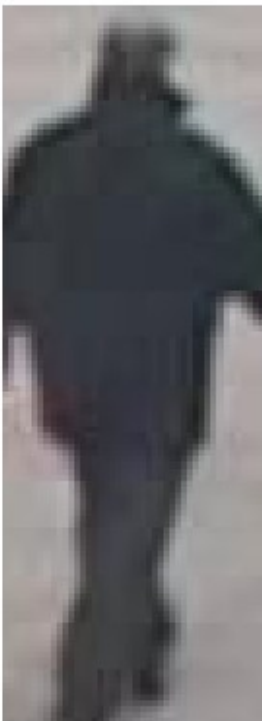
View Image Labels and Shape

```
plt.figure()  
plt.subplot(1, 2, 1)  
steak_img = view_random_image(f"{dataset_train}", "male")  
plt.subplot(1, 2, 2)  
pizza_img = view_random_image(f"{dataset_train}", "female")
```

```
Image shape: (121, 44, 3)
```

```
Image shape: (313, 118, 3)
```

male



female



Create train and test data generators and rescale the data

```
train_datagen = ImageDataGenerator(rescale=1/255.)  
test_datagen = ImageDataGenerator(rescale=1/255.)
```

Turn it into batches

```
train_data = train_datagen.flow_from_directory(directory=train_dir,  
                                                target_size=(224, 224),  
                                                class_mode='binary',  
                                                batch_size=32)  
  
test_data = test_datagen.flow_from_directory(directory=test_dir,  
                                              target_size=(224, 224),  
                                              class_mode='binary',  
                                              batch_size=32)
```

```
Found 6000 images belonging to 2 classes.  
Found 2000 images belonging to 2 classes.
```

Get a sample of the training data batch

```
images, labels = next(train_data)  
len(images), len(labels)  
  
(32, 32)
```

Check lengths of training and test data generators

```
len(train_data), len(test_data)  
  
(188, 63)
```

Plot the validation and training data separately

```
def plot_loss_curves(history):  
    loss = history.history['loss']  
    val_loss = history.history['val_loss']  
  
    accuracy = history.history['accuracy']  
    val_accuracy = history.history['val_accuracy']  
  
    epochs = range(len(history.history['loss']))  
  
    # Plot loss  
    plt.plot(epochs, loss, label='training_loss')  
    plt.plot(epochs, val_loss, label='val_loss')  
    plt.title('Loss')
```

```

plt.xlabel('Epochs')
plt.legend()

# Plot accuracy
plt.figure()
plt.plot(epochs, accuracy, label='training_accuracy')
plt.plot(epochs, val_accuracy, label='val_accuracy')
plt.title('Accuracy')
plt.xlabel('Epochs')
plt.legend();

```

Create ImageDataGenerator training instance with data augmentation

```

train_datagen_augmented = ImageDataGenerator(rescale=1/255.,
                                              rotation_range=20, #
rotate the image slightly between 0 and 20 degrees (note: this is an
int not a float)
                                              shear_range=0.2, # shear
the image
                                              zoom_range=0.2, # zoom
into the image
                                              width_shift_range=0.2, #
shift the image width ways
                                              height_shift_range=0.2, #
shift the image height ways
                                              horizontal_flip=True) #
flip the image on the horizontal axis

# Create ImageDataGenerator training instance without data
augmentation
train_datagen = ImageDataGenerator(rescale=1/255.)

# Create ImageDataGenerator test instance without data augmentation
test_datagen = ImageDataGenerator(rescale=1/255.)

```

Import data and augment it from training directory

```

print("Augmented training images:")
train_data_augmented =
train_datagen_augmented.flow_from_directory(train_dir,

target_size=(224, 224),

batch_size=32,

class_mode='binary',

shuffle=False)

```

```

# Create non-augmented data batches
print("Non-augmented training images:")
train_data = train_datagen.flow_from_directory(train_dir,
                                                target_size=(224, 224),
                                                batch_size=32,
                                                class_mode='binary',
                                                shuffle=False)

print("Unchanged test images:")
test_data = test_datagen.flow_from_directory(test_dir,
                                              target_size=(224, 224),
                                              batch_size=32,
                                              class_mode='binary')

Augmented training images:
Found 6000 images belonging to 2 classes.
Non-augmented training images:
Found 6000 images belonging to 2 classes.
Unchanged test images:
Found 2000 images belonging to 2 classes.

# Get data batch samples
images, labels = next(train_data)
augmented_images, augmented_labels = next(train_data_augmented)

# Show original image and augmented image
random_number = random.randint(0, 31) # Size of Batch is 32
plt.imshow(images[random_number])
plt.title(f"Original image")
plt.axis(False)
plt.figure()
plt.imshow(augmented_images[random_number])
plt.title(f"Augmented image")
plt.axis(False);

```

Original image



Augmented image



Import data and augment it from directories

```
train_data_augmented_shuffled =  
train_datagen_augmented.flow_from_directory(train_dir,  
  
target_size=(224, 224),  
  
batch_size=32,  
  
class_mode='binary',  
  
shuffle=True)  
  
Found 6000 images belonging to 2 classes.
```

Create the Model

```
# Create a CNN model (same as Tiny VGG but for binary classification -  
https://poloclub.github.io/cnn-explainer/ )  
model1 = Sequential([  
    Conv2D(10, 3, activation='relu', input_shape=(224, 224, 3)), # same  
    input shape as our images  
    Conv2D(10, 3, activation='relu'),  
    MaxPool2D(),  
    Conv2D(10, 3, activation='relu'),  
    Conv2D(10, 3, activation='relu'),  
    MaxPool2D(),  
    Flatten(),  
    Dropout(0.5),  
    Dense(1, activation='sigmoid')  
)  
  
# Compile the model  
model1.compile(loss="binary_crossentropy",  
               optimizer=tf.keras.optimizers.Adam(),  
               metrics=["accuracy"])  
  
# Fit the model  
history_8 = model1.fit(train_data_augmented_shuffled,  
                      epochs=13,  
  
steps_per_epoch=len(train_data_augmented_shuffled),  
                  validation_data=test_data,  
                  validation_steps=len(test_data))  
  
Epoch 1/13  
188/188 _____ 83s 436ms/step - accuracy: 0.5245 - loss:  
0.6923 - val_accuracy: 0.5310 - val_loss: 0.6907  
Epoch 2/13
```

```

188/188 _____ 86s 456ms/step - accuracy: 0.5537 - loss:
0.6858 - val_accuracy: 0.5635 - val_loss: 0.6836
Epoch 3/13
188/188 _____ 87s 463ms/step - accuracy: 0.5543 - loss:
0.6863 - val_accuracy: 0.5640 - val_loss: 0.6818
Epoch 4/13
188/188 _____ 87s 465ms/step - accuracy: 0.5588 - loss:
0.6845 - val_accuracy: 0.5810 - val_loss: 0.6769
Epoch 5/13
188/188 _____ 88s 469ms/step - accuracy: 0.5823 - loss:
0.6753 - val_accuracy: 0.6045 - val_loss: 0.6715
Epoch 6/13
188/188 _____ 88s 465ms/step - accuracy: 0.5940 - loss:
0.6732 - val_accuracy: 0.6290 - val_loss: 0.6599
Epoch 7/13
188/188 _____ 87s 464ms/step - accuracy: 0.5957 - loss:
0.6677 - val_accuracy: 0.6555 - val_loss: 0.6432
Epoch 8/13
188/188 _____ 87s 465ms/step - accuracy: 0.6052 - loss:
0.6640 - val_accuracy: 0.6460 - val_loss: 0.6593
Epoch 9/13
188/188 _____ 93s 496ms/step - accuracy: 0.6160 - loss:
0.6633 - val_accuracy: 0.5485 - val_loss: 0.7118
Epoch 10/13
188/188 _____ 93s 495ms/step - accuracy: 0.6020 - loss:
0.6642 - val_accuracy: 0.6510 - val_loss: 0.6330
Epoch 11/13
188/188 _____ 89s 470ms/step - accuracy: 0.6167 - loss:
0.6573 - val_accuracy: 0.6295 - val_loss: 0.6578
Epoch 12/13
188/188 _____ 88s 469ms/step - accuracy: 0.6277 - loss:
0.6518 - val_accuracy: 0.6555 - val_loss: 0.6369
Epoch 13/13
188/188 _____ 90s 480ms/step - accuracy: 0.6288 - loss:
0.6468 - val_accuracy: 0.6305 - val_loss: 0.6363

```

Check model1 architecture

```

# Check model1 architecture (same as model_1)
model1.summary()

```

```

Model: "sequential_1"

```

Layer (type)	Output Shape	
Param #		

conv2d_4 (Conv2D)	(None, 222, 222, 10)	
280		
conv2d_5 (Conv2D)	(None, 220, 220, 10)	
910		
max_pooling2d_2 (MaxPooling2D)	(None, 110, 110, 10)	
0		
conv2d_6 (Conv2D)	(None, 108, 108, 10)	
910		
conv2d_7 (Conv2D)	(None, 106, 106, 10)	
910		
max_pooling2d_3 (MaxPooling2D)	(None, 53, 53, 10)	
0		
flatten_1 (Flatten)	(None, 28090)	
0		
dropout_1 (Dropout)	(None, 28090)	
0		
dense_1 (Dense)	(None, 1)	
28,091		

Total params: 93,305 (364.48 KB)

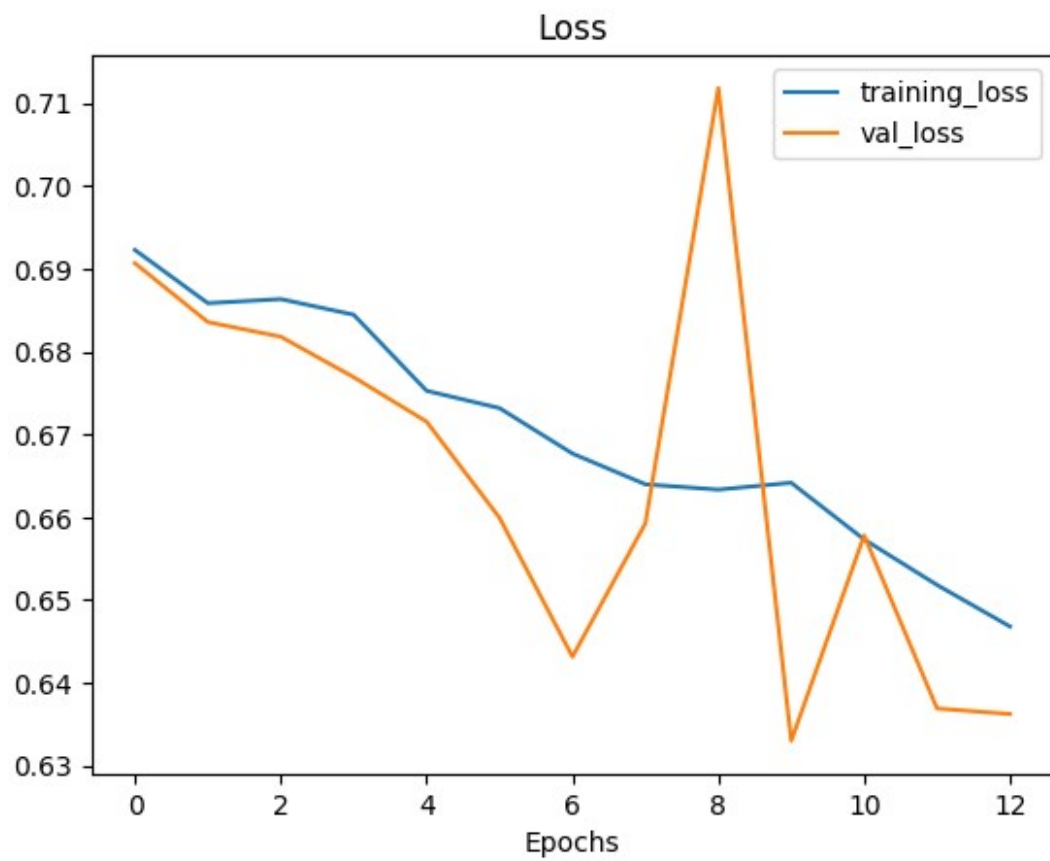
Trainable params: 31,101 (121.49 KB)

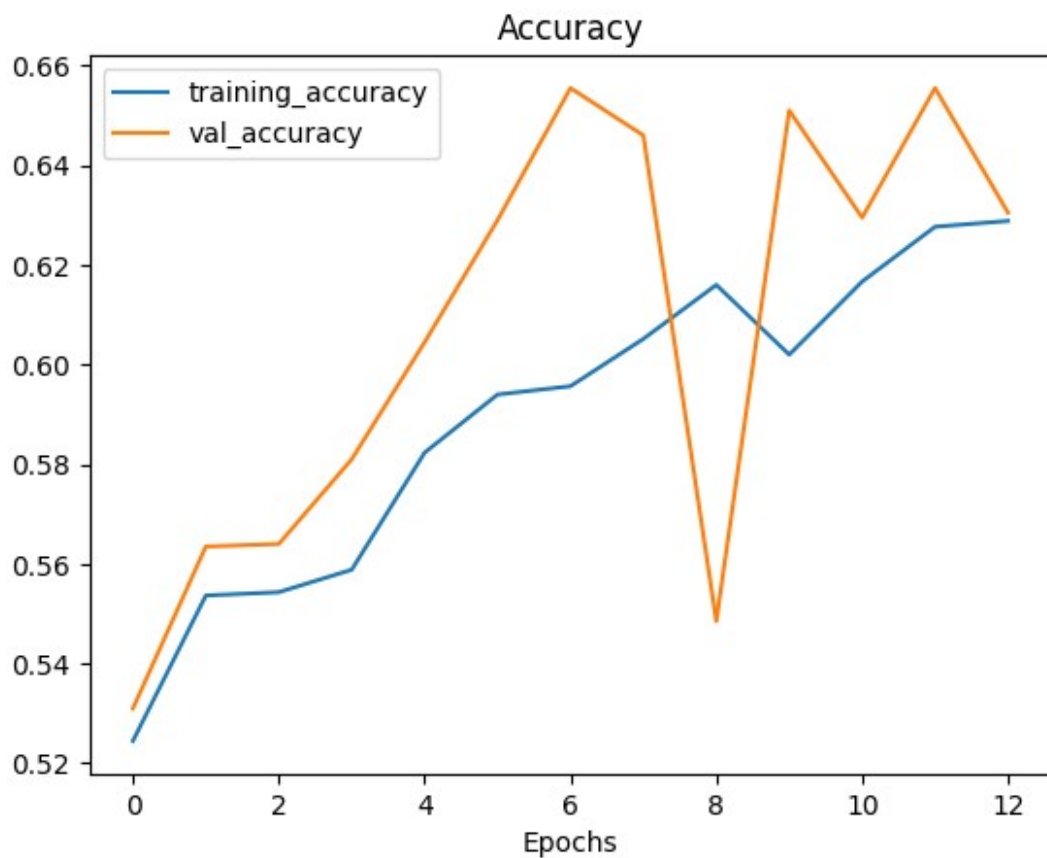
Non-trainable params: 0 (0.00 B)

Optimizer params: 62,204 (242.99 KB)

Plot Loss and Accuracy

```
plot_loss_curves(history_8)
```





View our example image

```
male = mpimg.imread(f"{dataset_test}/male/0_8.jpg")
plt.imshow(male)
plt.axis(False)

(np.float64(-0.5), np.float64(144.5), np.float64(259.5), np.float64(-0.5))
```



```
male.shape
```

```
(260, 145, 3)
```

Create a function to import an image and resize it.

```
def load_and_prep_image(filename, img_shape=224):  
    """  
    Reads an image from filename, turns it into a tensor  
    and reshapes it to (img_shape, img_shape, colour_channel).  
    """  
    # Read in target file (an image)  
    img = tf.io.read_file(filename)  
  
    # Decode the read file into a tensor & ensure 3 colour channels  
    # (our model is trained on images with 3 colour channels and  
    sometimes images have 4 colour channels)  
    img = tf.image.decode_image(img, channels=3)  
  
    # Resize the image (to the same size our model was trained on)  
    img = tf.image.resize(img, size = [img_shape, img_shape])  
  
    # Rescale the image (get all values between 0 and 1)  
    img = img/255.  
    return img
```

Load in and preprocess our custom image

```
male = load_and_prep_image(f"{dataset_test}/male/00910_male_fore.jpg")
male
```

```
<tf.Tensor: shape=(224, 224, 3), dtype=float32, numpy=
array([[ [0.53333336, 0.4117647 , 0.44313726],
        [0.53333336, 0.4117647 , 0.44313726],
        [0.5383754 , 0.4159664 , 0.44985995],
        ...,
        [0.69523877, 0.803082 , 0.7366954 ],
        [0.7490196 , 0.8627451 , 0.7921569 ],
        [0.7490196 , 0.8627451 , 0.7921569 ]],

        [ [0.50112045, 0.37815127, 0.40252098],
        [0.50112045, 0.37815127, 0.40252098],
        [0.5079632 , 0.3832533 , 0.41194478],
        ...,
        [0.7001407 , 0.7999807 , 0.7402968 ],
        [0.7518207 , 0.8557423 , 0.7921569 ],
        [0.7518207 , 0.8557423 , 0.7921569 ]],

        [ [0.4495798 , 0.32436976, 0.337535 ],
        [0.4495798 , 0.32436976, 0.337535 ],
        [0.4593037 , 0.33091235, 0.3512805 ],
        ...,
        [0.7079838 , 0.7950186 , 0.746059 ],
        [0.75630254, 0.84453785, 0.7921569 ],
        [0.75630254, 0.84453785, 0.7921569 ]],

        ...,

        [ [0.4369755 , 0.520449 , 0.49411854],
        [0.4369755 , 0.520449 , 0.49411854],
        [0.4371556 , 0.5213493 , 0.49129745],
        ...,
        [0.4865347 , 0.5782316 , 0.5453787 ],
        [0.49103647, 0.58375376, 0.5537819 ],
        [0.49103647, 0.58375376, 0.5537819 ]],

        [ [0.5131657 , 0.60560274, 0.58823586],
        [0.5131657 , 0.60560274, 0.58823586],
        [0.51478636, 0.60698324, 0.58973646],
        ...,
        [0.49085647, 0.60128075, 0.58347374],
        [0.49775913, 0.6106444 , 0.5918769 ],
        [0.49775913, 0.6106444 , 0.5918769 ]],

        [ [0.56078434, 0.65882355, 0.64705884],
        [0.56078434, 0.65882355, 0.64705884],
```

```

[0.5633053 , 0.6605042 , 0.6512605 ],
...,
[0.49355754, 0.6156864 , 0.60728306],
[0.5019608 , 0.627451  , 0.6156863 ],
[0.5019608 , 0.627451  , 0.6156863 ]]],
shape=(224, 224, 3), dtype=float32)>

```

Add an extra axis

```

print(f"Shape before new dimension: {male.shape}")
male = tf.expand_dims(male, axis=0) # add an extra dimension at axis 0

```

```

print(f"Shape after new dimension: {male.shape}")
male

```

```

Shape before new dimension: (224, 224, 3)
Shape after new dimension: (1, 224, 224, 3)

```

```

<tf.Tensor: shape=(1, 224, 224, 3), dtype=float32, numpy=
array([[[[0.53333336, 0.4117647 , 0.44313726],
          [0.53333336, 0.4117647 , 0.44313726],
          [0.5383754 , 0.4159664 , 0.44985995],
          ...,
          [0.69523877, 0.803082  , 0.7366954 ],
          [0.7490196 , 0.8627451 , 0.7921569 ],
          [0.7490196 , 0.8627451 , 0.7921569 ]],

         [[0.50112045, 0.37815127, 0.40252098],
          [0.50112045, 0.37815127, 0.40252098],
          [0.5079632 , 0.3832533 , 0.41194478],
          ...,
          [0.7001407 , 0.7999807 , 0.7402968 ],
          [0.7518207 , 0.8557423 , 0.7921569 ],
          [0.7518207 , 0.8557423 , 0.7921569 ]],

         [[0.4495798 , 0.32436976, 0.337535  ],
          [0.4495798 , 0.32436976, 0.337535  ],
          [0.4593037 , 0.33091235, 0.3512805 ],
          ...,
          [0.7079838 , 0.7950186 , 0.746059  ],
          [0.75630254, 0.84453785, 0.7921569 ],
          [0.75630254, 0.84453785, 0.7921569 ]],

         ...,

         [[0.4369755 , 0.520449  , 0.49411854],
          [0.4369755 , 0.520449  , 0.49411854],
          [0.4371556 , 0.5213493 , 0.49129745],
          ...,
          [0.4865347 , 0.5782316 , 0.5453787 ],

```



```

[[0.49103647, 0.58375376, 0.5537819 ],
 [0.49103647, 0.58375376, 0.5537819 ]],

[[0.5131657 , 0.60560274, 0.58823586],
 [0.5131657 , 0.60560274, 0.58823586],
 [0.51478636, 0.60698324, 0.58973646],
 ...,
 [0.49085647, 0.60128075, 0.58347374],
 [0.49775913, 0.6106444 , 0.5918769 ],
 [0.49775913, 0.6106444 , 0.5918769 ]],

[[0.56078434, 0.65882355, 0.64705884],
 [0.56078434, 0.65882355, 0.64705884],
 [0.5633053 , 0.6605042 , 0.6512605 ],
 ...,
 [0.49355754, 0.6156864 , 0.60728306],
 [0.5019608 , 0.627451 , 0.6156863 ],
 [0.5019608 , 0.627451 , 0.6156863 ]]],
shape=(1, 224, 224, 3), dtype=float32)>

```

Make a prediction on custom image tensor

```

pred = model1.predict(male)
pred

1/1 ————— 0s 57ms/step

array([[0.49081483]], dtype=float32)

```

Define prediction and plot

```

def pred_and_plot(model, filename, class_names):
    # Import the target image and preprocess it
    select_class = random.choice(class_names)
    name = f"{filename}{select_class}/"
    file = f"{name}{random.sample(os.listdir(name), 1)[0]}"

    img = load_and_prep_image(file)

    # Make a prediction
    pred = model.predict(tf.expand_dims(img, axis=0))
    # 1. Get the raw probability
    prob = pred[0][0]

    # 2. Determine the predicted class (0 or 1)
    predicted_class = 1 if prob > 0.5 else 0

    # 3. Calculate Confidence (how "sure" the model is)
    # If prob is 0.9, confidence is 0.9. If prob is 0.1, confidence is 0.9.

```

```

confidence = prob if predicted_class == 1 else (1 - prob)

print(f"Predicted Class: {predicted_class}")
print(f"Confidence: {confidence:.2%}")

# Get the predicted class
pred_class = class_names[int(tf.round(pred)[0][0])]

# Plot the image and predicted class
plt.imshow(img)
plt.title(f"Prediction {confidence:.2%}: {pred_class}")
plt.axis(False)

```

Predict Random Images

```
pred_and_plot(model1, dataset_test, class_names)
```

```

1/1 ————— 0s 26ms/step
Predicted Class: 0
Confidence: 67.13%

```

Prediction 67.13%: female



```
pred_and_plot(model1, dataset_test, class_names)
```

1/1 ————— 0s 25ms/step
Predicted Class: 0
Confidence: 72.20%

Prediction 72.20%: female



```
pred_and_plot(model1, dataset_test, class_names)
```

1/1 ————— 0s 22ms/step
Predicted Class: 1
Confidence: 68.07%

Prediction 68.07%: male



```
pred_and_plot(model1, dataset_test, class_names)
```

1/1 ————— 0s 25ms/step

Predicted Class: 1

Confidence: 70.85%

Prediction 70.85%: male

